

System Documentation

January 2026

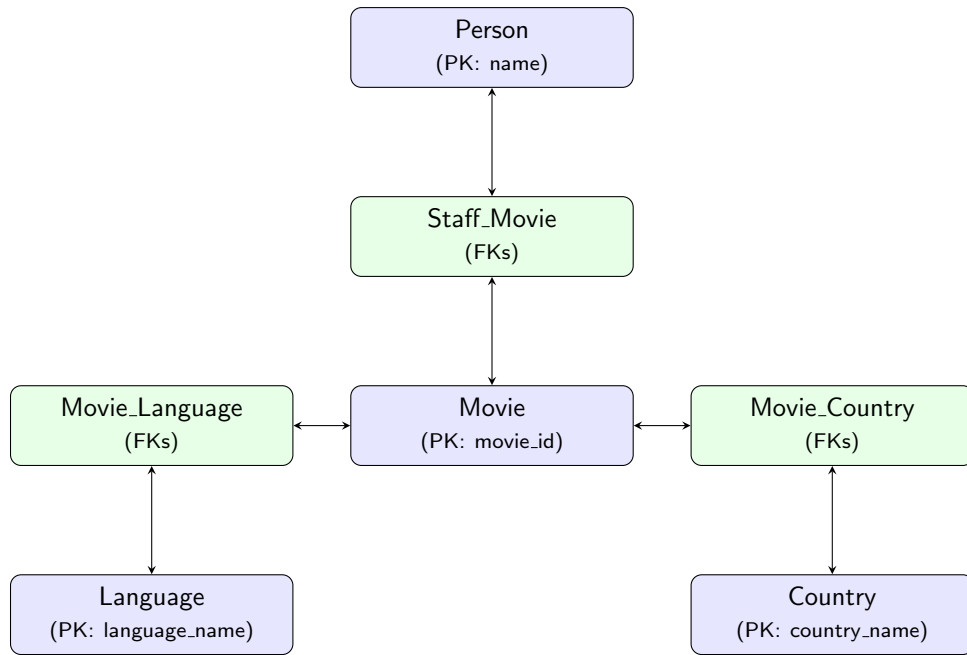
1 Database Schema Description

Our movie database is designed using a normalized relational schema to ensure data integrity and efficient querying. The schema consists of the following tables:

- **Movie:** The central entity table.
 - **Columns:** `movie_id` (Primary Key, Auto Increment), `title`, `average_rating`, `release_date`, `budget` (BigInt), `revenue` (BigInt), `runtime` (Float), and `meta_score` (Int).
 - **Indices:** A FULLTEXT index is defined on the `title` column (`idx_movie_title`) to support efficient keyword-based searching.
- **Person:** A lookup table representing all individuals involved in the film industry.
 - **Columns:** `name` (Primary Key). This ensures that each person (actor, director, or writer) is stored uniquely.
- **Staff_Movie:** A junction table representing the many-to-many relationship between Person and Movie.
 - **Columns:** `person_name`, `movie_id`, and `role`.
 - **Role Definition:** The `role` column is an ENUM('actor', 'writer', 'director'), allowing us to distinguish the type of contribution a person made to a specific movie.
 - **Indices:** A FULLTEXT index on `person_name` (`idx_person_name`) facilitates efficient searching for filmographies.
 - **Foreign Keys:** References `Person(name)` and `Movie(movie_id)`.
- **Country:** A lookup table for countries.
 - **Columns:** `country_name` (Primary Key).
- **Movie_Country:** A junction table linking movies to their countries of origin.
 - **Foreign Keys:** References `Movie(movie_id)` and `Country(country_name)`.
- **Language:** A lookup table for languages.
 - **Columns:** `language_name` (Primary Key).
- **Movie_Language:** A junction table linking movies to available languages.
 - **Foreign Keys:** References `Movie(movie_id)` and `Language(language_name)`.

1.1 Schema Diagram

The following diagram illustrates the relationships between the database tables.



2 Reasoning for Database Design

2.1 Normalization & Efficiency

We chose a highly normalized design (Third Normal Form) rather than storing lists of actors or languages as comma-separated strings within the Movie table.

- **Efficiency:** Storing data in junction tables (**Staff_Movie**, **Movie_Language**) allows us to use standard SQL JOIN operations. This is significantly faster for queries like “Find all movies by Tom Hanks” compared to scanning string columns with LIKE `'%...%'`.
- **Data Integrity:** Unique constraints on the Person, Country, and Language tables prevent typo duplication (e.g., storing “USA” and “U.S.A.” as different entities).
- **Scalability:** The design easily supports movies with hundreds of cast members or languages without hitting column character limits.

2.2 Drawbacks & Mitigation

- **Drawback:** The normalized design requires more complex queries involving multiple JOINS to retrieve full movie details.
- **Mitigation:** We mitigated this complexity by creating dedicated query functions in Python that handle these joins transparently.

3 Database Optimizations (Indices)

To optimize performance for the expected workload (text-heavy searches), we implemented Full-Text Indices in addition to standard Primary Key indices.

1. `idx_movie_title` on Movie (**title**):

- **Purpose:** Supports Query 1 (Search by Title).
- **Why:** Standard B-Tree indices are inefficient for searching substrings. A Full-Text index allows the database to use natural language matching algorithms, which are faster and more relevant for user searches.

2. `idx_person_name` on `Staff_Movie` (`person_name`):

- **Purpose:** Supports Query 2 (Search by Staff).
- **Why:** Users frequently search for actors or directors. Indexing this column ensures that retrieving a person’s filmography is an instantaneous lookup operation rather than a full table scan.

4 Main Queries Description

We implemented five specific queries to demonstrate the database’s capabilities:

1. **Title Keyword Search (Full-Text)**

- **Description:** Finds movies whose titles match a given keyword using natural language mode.
- **Support:** Leverages the `MATCH()` `AGAINST()` syntax and the `idx_movie_title` index.

2. **Staff Filmography Search (Full-Text)**

- **Description:** Retrieves all movies associated with a specific actor, director, or writer.
- **Support:** Uses a `JOIN` between `Movie` and `Staff_Movie`, optimized by the `idx_person_name` index.

3. **Revenue Trend Analysis (Aggregation)**

- **Description:** Groups movies by their release year (`YEAR(release_date)`) and calculates the total revenue for each year.
- **Purpose:** Allows analysts to track industry financial growth over time.

4. **High-Performance Actors (Nested Query + Aggregation)**

- **Description:** Identifies actors whose personal average movie rating is higher than the global average rating of all movies.
- **Complexity:** Uses a nested subquery to calculate the global average, then `GROUP BY` actor name with a `HAVING` clause to filter the results.

5. **Global Blockbuster Filter (Exists + Case)**

- **Description:** Finds high-budget movies (> \$100M) that are multi-lingual. It includes a custom boolean column checking if “Arabic” is one of the available languages.
- **Complexity:** Uses a correlated `EXISTS` subquery within a `CASE` statement, which is highly efficient as it stops scanning the `Movie_Language` table as soon as a match is found.

5 Code Structure

The project is structured into four main Python modules located in the `src/` directory, adhering to the assignment requirements:

- `src/db_config.py`
 - **Purpose:** Centralized configuration file.
 - **Details:** Stores database credentials (host, user, password) and connection flags. It ensures consistent connection settings across all scripts and enables the `use_pure=True` flag required for the university server authentication.
- `src/create_db_script.py`
 - **Purpose:** Database initialization (DDL).
 - **Key Function:** `create_database()`.
 - **Logic:** Connects to the server, drops existing tables to ensure a clean state, creates the new normalized schema tables, and establishes Full-Text indices.

- `src/api_data_retrieve.py`
 - **Purpose:** Data extraction and loading (ETL).
 - **Key Function:** `insert_data(csv_file_path)`.
 - **Logic:** Reads the raw CSV dataset using pandas. It cleanses data (parsing dates, converting currency strings to integers) and iteratively inserts records into the `Movie`, `Person`, `Staff_Movie`, `Country`, and `Language` tables while handling duplicates via `INSERT IGNORE`.
- `src/queries_db_script.py`
 - **Purpose:** Encapsulation of SQL logic.
 - **Key Functions:** `query_1(keyword)` through `query_5()`.
 - **Logic:** These functions return the raw SQL query strings. A helper function `execute_query()` handles the connection, cursor execution, and result formatting using the `tabulate` library.
- `src/queries_execution.py`
 - **Purpose:** Main execution entry point.
 - **Logic:** Imports the query functions from `queries_db_script.py` and executes them with sample parameters (e.g., searching for “Star” or “Hanks”) to demonstrate system functionality in the console.